

Neural Network Prediction of Climate-Related Supply Chain Risk: AER-Style Figures

The figures use the original Stata replication data. The prediction label is $\mathbb{1}\{\sum_{h=1}^4 Ter_q_qh > 0\}$, defined on observations with all four future termination variables observed. The neural network is a two-hidden-layer MLP classifier trained only on pre-holdout observations. No simulated outcome data are used.

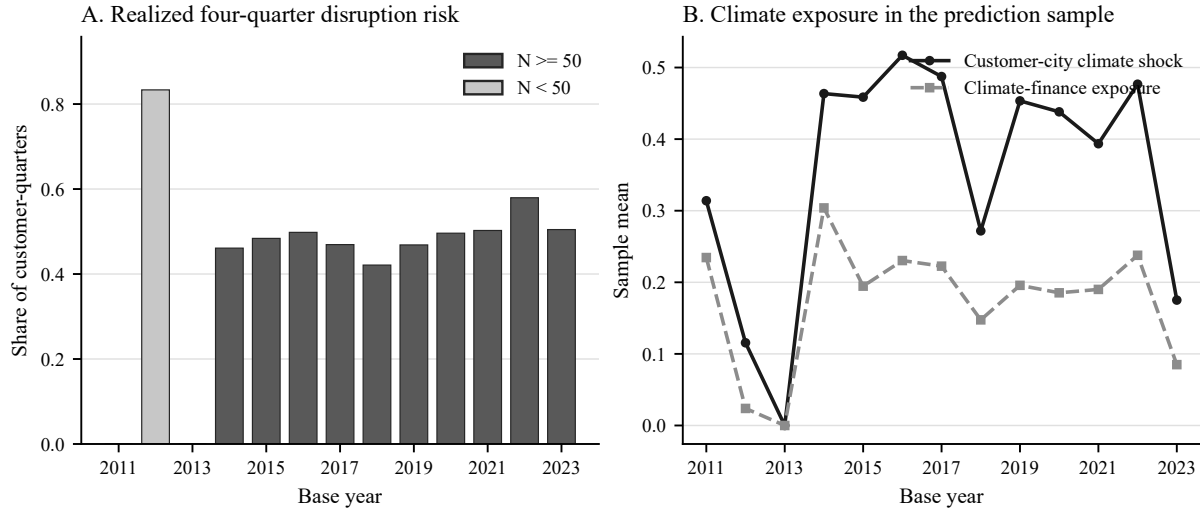


Figure 1. Prediction sample and climate exposure. Notes: Panel A reports the annual share of customer-quarter observations with a positive increase in relationship terminations over the following four quarters. Panel B reports the sample mean of the customer-city extreme-weather variable and the climate-finance exposure variable. Light bars flag sparse early-year cells with fewer than 50 observations.

Python code for Figure 1.

```
import matplotlib.pyplot as plt
from matplotlib.patches import Patch
import pandas as pd

from common import DATA_DIR, load_reorganization_data, save_figure

def main():
    data = load_reorganization_data()
    annual = (
        data.groupby("year")
        .agg(
            observations=("BreakRisk4Q", "size"),
            break_risk=("BreakRisk4Q", "mean"),
            affected=("Affected", "mean"),
            climate_city=("Climate_CityC", "mean"),
        )
    )
```

```

    )
    .reset_index()
)
annual.to_csv(DATA_DIR / "figure_1_sample_facts.csv", index=False)

fig, axes = plt.subplots(1, 2, figsize=(7.0, 3.0), constrained_layout=True)

colors = ["0.35" if n >= 50 else "0.78" for n in annual["observations"]]
axes[0].bar(
    annual["year"],
    annual["break_risk"],
    width=0.75,
    color=colors,
    edgecolor="0.15",
    linewidth=0.5,
)
axes[0].set_title("A. Realized four-quarter disruption risk", loc="left")
axes[0].set_ylabel("Share of customer-quarters")
axes[0].set_xlabel("Base year")
axes[0].set_ylim(0, max(0.85, annual["break_risk"].max() * 1.15))
axes[0].set_xticks(annual["year"][:,2])
axes[0].legend(
    handles=[
        Patch(facecolor="0.35", edgecolor="0.15", label="N >= 50"),
        Patch(facecolor="0.78", edgecolor="0.15", label="N < 50"),
    ],
    frameon=False,
    loc="upper right",
)

axes[1].plot(
    annual["year"],
    annual["climate_city"],
    color="0.10",
    marker="o",
    markersize=3,
    label="Customer-city climate shock",
)
axes[1].plot(
    annual["year"],
    annual["affected"],
    color="0.55",
    linestyle="--",
    marker="s",
    markersize=3,
    label="Climate-finance exposure",
)
axes[1].set_title("B. Climate exposure in the prediction sample", loc="left")
axes[1].set_ylabel("Sample mean")
axes[1].set_xlabel("Base year")
axes[1].set_xticks(annual["year"][:,2])
axes[1].legend(frameon=False, loc="upper right")

for ax in axes:
    ax.grid(axis="y", color="0.88", linewidth=0.6)
    ax.set_axisbelow(True)

save_figure(fig, "fig1_sample_facts")

if __name__ == "__main__":
    main()

```

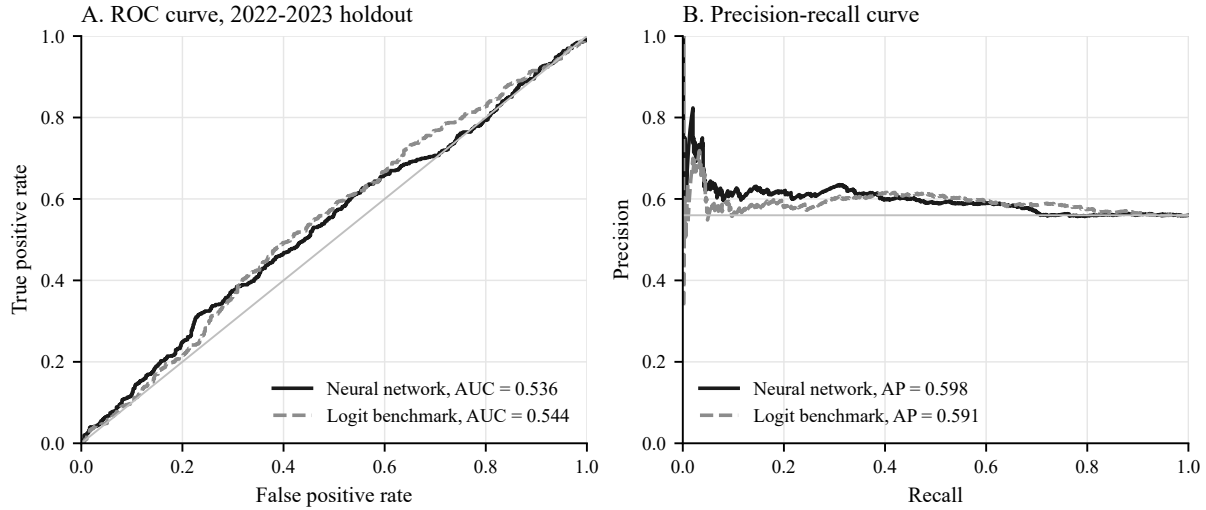


Figure 2. Out-of-sample predictive performance. Notes: The holdout period is 2022-2023. The logit curve is included only as a transparent benchmark for the neural network, using the same input variables and train-test split.

Python code for Figure 2.

```
import matplotlib.pyplot as plt
import pandas as pd

from common import (
    DATA_DIR,
    model_holdout,
    roc_pr_curves,
    save_figure,
)

def main():
    _, _, mlp_test, mlp_metrics = model_holdout(cutoff_year=2021, estimator="mlp")
    _, _, logit_test, logit_metrics = model_holdout(cutoff_year=2021, estimator="logit")

    y_true = mlp_test["BreakRisk4Q"]
    fpr_mlp, tpr_mlp, prec_mlp, rec_mlp = roc_pr_curves(y_true, mlp_test["pred"])
    fpr_logit, tpr_logit, prec_logit, rec_logit = roc_pr_curves(
        logit_test["BreakRisk4Q"], logit_test["pred"]
    )

    pd.DataFrame([mlp_metrics, logit_metrics]).to_csv(
        DATA_DIR / "figure_2_holdout_metrics.csv", index=False
    )

    fig, axes = plt.subplots(1, 2, figsize=(7.0, 3.0), constrained_layout=True)

    axes[0].plot(
        fpr_mlp,
        tpr_mlp,
        color="0.10",
        label=f"Neural network, AUC = {mlp_metrics['auc']:.3f}",
    )
    axes[0].plot(
        fpr_logit,
        tpr_logit,
        color="0.55",
        linestyle="--",
        label=f"Logit benchmark, AUC = {logit_metrics['auc']:.3f}",
    )
```

```

)
axes[0].plot([0, 1], [0, 1], color="0.75", linewidth=0.8)
axes[0].set_title("A. ROC curve, 2022-2023 holdout", loc="left")
axes[0].set_xlabel("False positive rate")
axes[0].set_ylabel("True positive rate")
axes[0].legend(frameon=False, loc="lower right")

axes[1].plot(
    rec_mlp,
    prec_mlp,
    color="0.10",
    label=f"Neural network, AP = {mlp_metrics['average_precision']:.3f}",
)
axes[1].plot(
    rec_logit,
    prec_logit,
    color="0.55",
    linestyle="--",
    label=f"Logit benchmark, AP = {logit_metrics['average_precision']:.3f}",
)
axes[1].axhline(y_true.mean(), color="0.75", linewidth=0.8)
axes[1].set_title("B. Precision-recall curve", loc="left")
axes[1].set_xlabel("Recall")
axes[1].set_ylabel("Precision")
axes[1].legend(frameon=False, loc="lower left")

for ax in axes:
    ax.set_xlim(0, 1)
    ax.set_ylim(0, 1)
    ax.grid(color="0.90", linewidth=0.6)
    ax.set_axisbelow(True)

save_figure(fig, "fig2_prediction_curves")

if __name__ == "__main__":
    main()

```

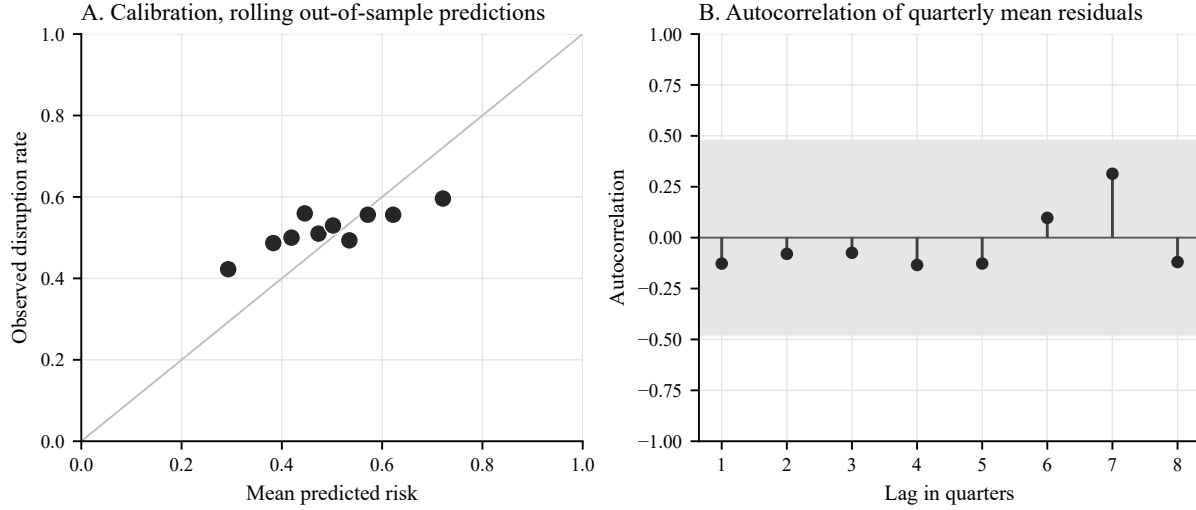


Figure 3. Calibration and residual diagnostics. Notes: Predictions are produced by a rolling one-year-ahead design. Panel A bins neural-network predictions into deciles. Panel B plots autocorrelations of quarterly mean residuals; the shaded band is $\pm 1.96/\sqrt{T}$.

Python code for Figure 3.

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from common import DATA_DIR, rolling_oos_predictions, save_figure

def autocorrelation(values, max_lag):
    x = np.asarray(values, dtype=float)
    x = x - x.mean()
    denom = np.dot(x, x)
    if denom == 0:
        return np.zeros(max_lag)
    return np.array([np.dot(x[lag:], x[:-lag]) / denom for lag in range(1, max_lag + 1)])

def main():
    predictions, metrics = rolling_oos_predictions(start_cutoff=2018, end_cutoff=2022)
    predictions.to_csv(DATA_DIR / "figure_3_rolling_predictions.csv", index=False)
    metrics.to_csv(DATA_DIR / "figure_3_rolling_metrics.csv", index=False)

    deciles = pd.qcut(predictions["pred"], 10, duplicates="drop")
    calibration = (
        predictions.assign(decile=deciles)
        .groupby("decile", observed=True)
        .agg(
            mean_pred=("pred", "mean"),
            observed=("BreakRisk4Q", "mean"),
            n=("BreakRisk4Q", "size"),
        )
        .reset_index(drop=True)
    )
    calibration.to_csv(DATA_DIR / "figure_3_calibration_bins.csv", index=False)

    quarterly = (
        predictions.groupby("yq")
        .agg(residual=("residual", "mean"), n=("residual", "size"))
        .reset_index()
        .sort_values("yq")
    )
```

```

)
max_lag = min(8, len(quarterly) - 2)
acf = autocorrelation(quarterly["residual"], max_lag)
acf_data = pd.DataFrame({"lag": np.arange(1, max_lag + 1), "acf": acf})
acf_data.to_csv(DATA_DIR / "figure_3_residual_acf.csv", index=False)
band = 1.96 / np.sqrt(len(quarterly))

fig, axes = plt.subplots(1, 2, figsize=(7.0, 3.0), constrained_layout=True)

axes[0].plot([0, 1], [0, 1], color="0.75", linewidth=0.8)
axes[0].scatter(
    calibration["mean_pred"],
    calibration["observed"],
    s=20 + 2 * np.sqrt(calibration["n"]),
    color="0.15",
    edgecolor="white",
    linewidth=0.5,
    zorder=3,
)
axes[0].set_title("A. Calibration, rolling out-of-sample predictions", loc="left")
axes[0].set_xlabel("Mean predicted risk")
axes[0].set_ylabel("Observed disruption rate")
axes[0].set_xlim(0, 1)
axes[0].set_ylim(0, 1)

axes[1].axhspan(-band, band, color="0.90", zorder=0)
axes[1].axhline(0, color="0.35", linewidth=0.8)
axes[1].vlines(acf_data["lag"], 0, acf_data["acf"], color="0.25", linewidth=1.2)
axes[1].scatter(acf_data["lag"], acf_data["acf"], color="0.15", s=18, zorder=3)
axes[1].set_title("B. Autocorrelation of quarterly mean residuals", loc="left")
axes[1].set_xlabel("Lag in quarters")
axes[1].set_ylabel("Autocorrelation")
axes[1].set_xticks(acf_data["lag"])
axes[1].set_ylim(-1, 1)

for ax in axes:
    ax.grid(color="0.90", linewidth=0.6)
    ax.set_axisbelow(True)

save_figure(fig, "fig3_calibration_residuals")

if __name__ == "__main__":
    main()

```

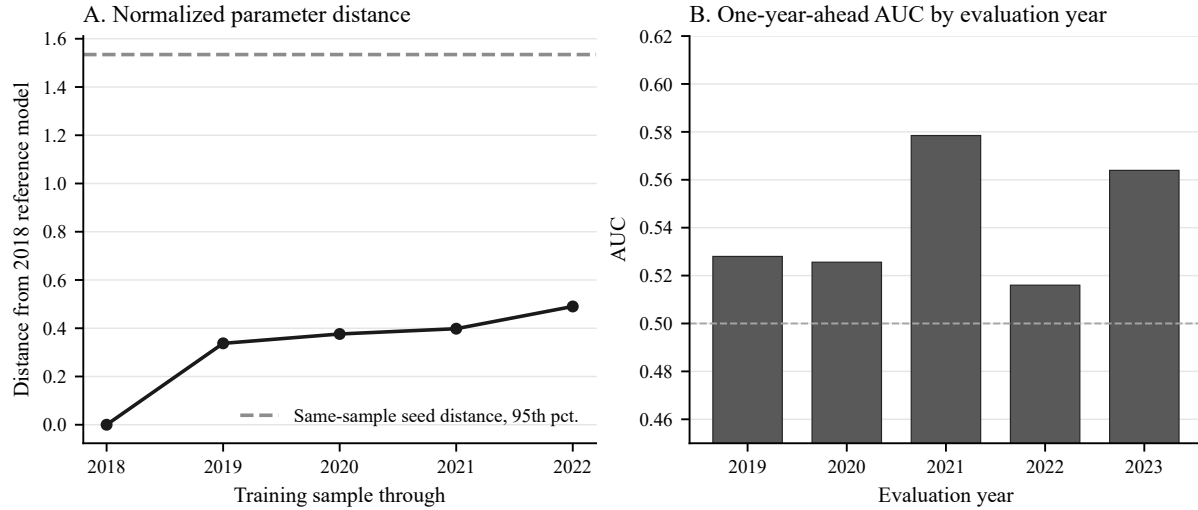


Figure 4. Model stability diagnostics. Notes: Panel A follows the parameter-distance logic of the model-stability framework. Distances are normalized Frobenius distances between MLP parameter vectors and the 2018 reference model. The dashed line is the 95th percentile of same-sample distances generated by changing only the random seed. Panel B reports the corresponding one-year-ahead AUC.

Python code for Figure 4.

```
import matplotlib.pyplot as plt

from common import DATA_DIR, save_figure, stability_diagnostics

def main():
    diagnostics, null = stability_diagnostics(reference_year=2018, last_cutoff=2022)
    diagnostics.to_csv(DATA_DIR / "figure_4_stability_diagnostics.csv", index=False)
    null.to_csv(DATA_DIR / "figure_4_same_sample_null_distances.csv", index=False)

    fig, axes = plt.subplots(1, 2, figsize=(7.0, 3.0), constrained_layout=True)

    axes[0].plot(
        diagnostics["train_cutoff"],
        diagnostics["distance_to_reference"],
        color="0.10",
        marker="o",
        markersize=4,
    )
    axes[0].axhline(
        diagnostics["null_q95"].iloc[0],
        color="0.55",
        linestyle="--",
        label="Same-sample seed distance, 95th pct.",
    )
    axes[0].set_title("A. Normalized parameter distance", loc="left")
    axes[0].set_xlabel("Training sample through")
    axes[0].set_ylabel("Distance from 2018 reference model")
    axes[0].legend(frameon=False, loc="lower right")

    axes[1].bar(
        diagnostics["eval_year"],
        diagnostics["auc"],
        color="0.35",
        edgecolor="0.15",
        linewidth=0.5,
        width=0.7,
    )
```

```

axes[1].axhline(0.5, color="0.65", linestyle="--", linewidth=0.8)
axes[1].set_title("B. One-year-ahead AUC by evaluation year", loc="left")
axes[1].set_xlabel("Evaluation year")
axes[1].set_ylabel("AUC")
axes[1].set_ylim(0.45, max(0.62, diagnostics["auc"].max() + 0.03))

for ax in axes:
    ax.grid(axis="y", color="0.90", linewidth=0.6)
    ax.set_axisbelow(True)

save_figure(fig, "fig4_model_stability")

if __name__ == "__main__":
    main()

```

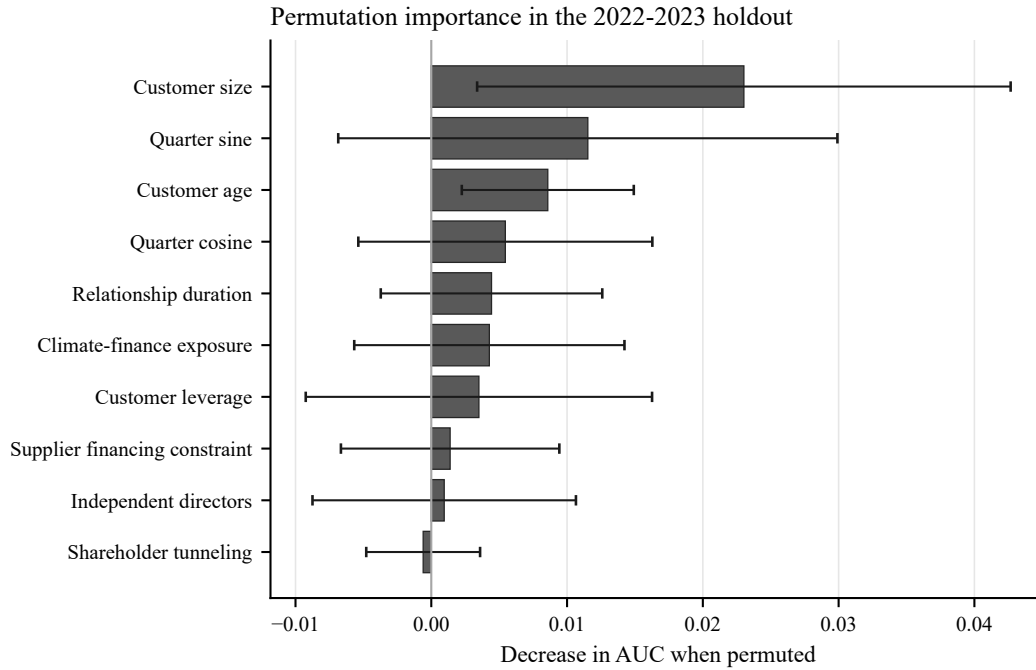


Figure 5. Permutation importance in the holdout sample. Notes: Each bar is the average decrease in holdout AUC when the feature is permuted 50 times. Error bars are 95 percent simulation intervals across permutations.

Python code for Figure 5.

```
import matplotlib.pyplot as plt
import pandas as pd
from sklearn.inspection import permutation_importance

from common import (
    DATA_DIR,
    FEATURE_LABELS,
    FEATURES,
    model_holdout,
    save_figure,
)

def main():
    model, _, test, metrics = model_holdout(cutoff_year=2021, estimator="mlp")
    result = permutation_importance(
        model,
        test[FEATURES],
        test["BreakRisk4Q"],
        scoring="roc_auc",
        n_repeats=50,
        random_state=42,
    )
    importance = pd.DataFrame(
        {
            "feature": FEATURES,
            "label": [FEATURE_LABELS.get(feature, feature) for feature in FEATURES],
            "mean_auc_loss": result.importances_mean,
            "sd_auc_loss": result.importances_std,
            "holdout_auc": metrics["auc"],
        }
    ).sort_values("mean_auc_loss", ascending=False)
```

```

importance.to_csv(DATA_DIR / "figure_5_permutation_importance.csv", index=False)
plot_data = importance.head(10).sort_values("mean_auc_loss")

fig, ax = plt.subplots(figsize=(5.6, 3.6), constrained_layout=True)
ax.barh(
    plot_data["label"],
    plot_data["mean_auc_loss"],
    xerr=1.96 * plot_data["sd_auc_loss"],
    color="0.35",
    edgecolor="0.15",
    linewidth=0.5,
    error_kw={"ecolor": "0.10", "linewidth": 0.8, "capsize": 2},
)
ax.axvline(0, color="0.65", linewidth=0.8)
ax.set_title("Permutation importance in the 2022-2023 holdout", loc="left")
ax.set_xlabel("Decrease in AUC when permuted")
ax.grid(axis="x", color="0.90", linewidth=0.6)
ax.set_axisbelow(True)

save_figure(fig, "fig5_permutation_importance")

if __name__ == "__main__":
    main()

```

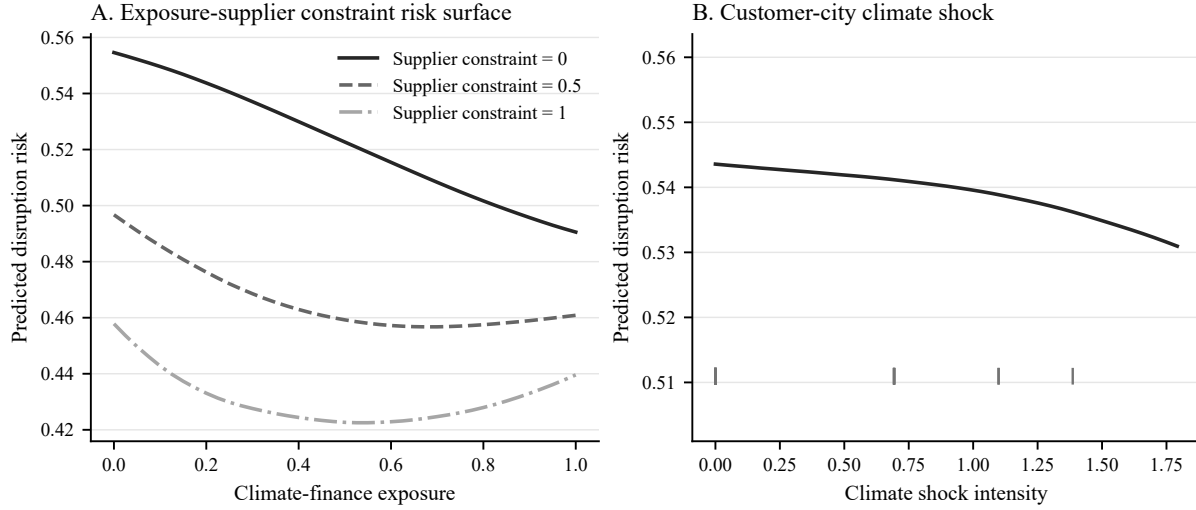


Figure 6. Model-implied supply chain risk surfaces. Notes: Curves average predicted risks over the 2022-2023 holdout covariate distribution. Panel A varies climate-finance exposure and supplier financing constraints while updating their interaction exactly as in the original data. Panel B varies customer-city climate shock intensity; tick marks show the empirical support of the holdout distribution.

Python code for Figure 6.

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from common import DATA_DIR, FEATURES, model_holdout, save_figure

def averaged_predictions(model, base, variable, grid, fixed_values=None):
    rows = []
    for value in grid:
        scenario = base.copy()
        scenario[variable] = value
        if fixed_values:
            for key, fixed_value in fixed_values.items():
                scenario[key] = fixed_value
        if "Affected" in scenario.columns and "S_FC" in scenario.columns:
            scenario["Affected_x_S_FC"] = scenario["Affected"] * scenario["S_FC"]
        pred = model.predict_proba(scenario[FEATURES])[:, 1].mean()
        rows.append({"grid_value": value, "predicted_risk": pred, **(fixed_values or {})})
    return pd.DataFrame(rows)

def main():
    model, train, test, _ = model_holdout(cutoff_year=2021, estimator="mlp")
    base = test[FEATURES].copy()

    affected_grid = np.linspace(0, 1, 41)
    sfc_levels = [0.0, 0.5, 1.0]
    affected_curves = pd.concat(
        [
            averaged_predictions(
                model,
                base,
                "Affected",
                affected_grid,
                fixed_values={"S_FC": sfc},
            ).assign(sfc_level=sfc)
            for sfc in sfc_levels
        ]
    )
```

```

    ],
    ignore_index=True,
)

climate_grid = np.linspace(test["Climate_CityC"].min(), test["Climate_CityC"].max(), 41)
climate_curve = averaged_predictions(model, base, "Climate_CityC", climate_grid)

affected_curves.to_csv(DATA_DIR / "figure_6_affected_surface.csv", index=False)
climate_curve.to_csv(DATA_DIR / "figure_6_climate_curve.csv", index=False)

fig, axes = plt.subplots(1, 2, figsize=(7.0, 3.0), constrained_layout=True)

styles = {0.0: ("0.15", "--"), 0.5: ("0.40", "--"), 1.0: ("0.65", "-.")}
for sfc in sfc_levels:
    subset = affected_curves[affected_curves["sfc_level"] == sfc]
    color, linestyle = styles[sfc]
    axes[0].plot(
        subset["grid_value"],
        subset["predicted_risk"],
        color=color,
        linestyle=linestyle,
        label=f"Supplier constraint = {sfc:g}",
    )
axes[0].set_title("A. Exposure-supplier constraint risk surface", loc="left")
axes[0].set_xlabel("Climate-finance exposure")
axes[0].set_ylabel("Predicted disruption risk")
axes[0].legend(frameon=False, loc="upper right")

axes[1].plot(
    climate_curve["grid_value"],
    climate_curve["predicted_risk"],
    color="0.15",
)
y_min = climate_curve["predicted_risk"].min() - 0.02
y_max = climate_curve["predicted_risk"].max() + 0.02
support = np.quantile(test["Climate_CityC"], np.linspace(0.05, 0.95, 19))
axes[1].plot(support, np.repeat(y_min, len(support)), "|", color="0.45", markersize=7)
axes[1].set_title("B. Customer-city climate shock", loc="left")
axes[1].set_xlabel("Climate shock intensity")
axes[1].set_ylabel("Predicted disruption risk")
axes[1].set_ylim(y_min - 0.01, y_max)

for ax in axes:
    ax.grid(axis="y", color="0.90", linewidth=0.6)
    ax.set_axisbelow(True)

save_figure(fig, "fig6_risk_surface")

if __name__ == "__main__":
    main()

```

Shared Python Utilities

The figure scripts above import the following shared functions for data loading, target construction, model training, prediction, and parameter-distance diagnostics.

```
from pathlib import Path
import warnings

import matplotlib as mpl
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import pyreadstat
from sklearn.exceptions import ConvergenceWarning
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    average_precision_score,
    brier_score_loss,
    precision_recall_curve,
    roc_auc_score,
    roc_curve,
)
from sklearn.neural_network import MLPClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

warnings.filterwarnings("ignore", category=ConvergenceWarning)

WORKSPACE = Path(__file__).resolve().parents[3]
OUT_DIR = WORKSPACE / "output" / "nn_supply_chain_figures"
FIG_DIR = OUT_DIR / "figures"
DATA_DIR = OUT_DIR / "data"
for directory in (FIG_DIR, DATA_DIR):
    directory.mkdir(parents=True, exist_ok=True)

FEATURES = [
    "Affected",
    "S_FC",
    "Affected_x_S_FC",
    "Climate_CityC",
    "C_Size",
    "C_Lev",
    "C_Inv",
    "C_Ppe",
    "C_Firmage",
    "C_Board",
    "C_Occupy",
    "C_Indep",
    "C_Frequency",
    "C_Duration",
    "year_c",
    "quarter_sin",
    "quarter_cos",
]

FEATURE_LABELS = {
    "Affected": "Climate-finance exposure",
    "S_FC": "Supplier financing constraint",
    "Affected_x_S_FC": "Exposure x supplier constraint",
    "Climate_CityC": "Customer-city climate shock",
    "C_Size": "Customer size",
    "C_Lev": "Customer leverage",
    "C_Inv": "Customer inventory",
    "C_Ppe": "Customer fixed assets",
    "C_Firmage": "Customer age",
    "C_Board": "Board size",
}
```

```

"C_Occupy": "Shareholder tunneling",
"C_Indep": "Independent directors",
"C_Frequency": "Trading frequency",
"C_Duration": "Relationship duration",
"year_c": "Calendar year",
"quarter_sin": "Quarter sine",
"quarter_cos": "Quarter cosine",
}

def set_aer_style():
    mpl.rcParams.update(
        {
            "figure.figsize": (6.8, 4.2),
            "figure.dpi": 150,
            "savefig.dpi": 300,
            "font.family": "serif",
            "font.serif": ["Times New Roman", "Times", "DejaVu Serif"],
            "font.size": 9,
            "axes.labelsize": 9,
            "axes.titlesize": 10,
            "legend.fontsize": 8,
            "xtick.labelsize": 8,
            "ytick.labelsize": 8,
            "axes.spines.top": False,
            "axes.spines.right": False,
            "axes.linewidth": 0.8,
            "xtick.major.width": 0.8,
            "ytick.major.width": 0.8,
            "lines.linewidth": 1.5,
            "pdf.fonttype": 42,
            "ps.fonttype": 42,
        }
    )

set_aer_style()

def find_dta(name_tokens):
    candidates = [
        path
        for path in WORKSPACE.rglob("*.dta")
        if all(token in path.name for token in name_tokens)
    ]
    if not candidates:
        raise FileNotFoundError(f"No Stata file matches tokens: {name_tokens}")
    candidates.sort(key=lambda path: (len(path.parts), len(str(path))))
    return candidates[0]

def stata_yq_to_year_quarter(yq_series):
    yq = yq_series.astype(int)
    year = 1960 + (yq // 4)
    quarter = (yq % 4) + 1
    return year, quarter

def load_reorganization_data():
    path = find_dta(["14_", "15_"])
    raw, meta = pyreadstat.read_dta(str(path))
    ter_cols = sorted(
        [column for column in raw.columns if column.startswith("Ter_q-")],
        key=lambda column: int(column.split("q")[-1]),
    )
    interaction = [
        column
        for column in raw.columns

```

```

        if column.startswith("Affected") and column != "Affected"
    ] [0]

    data = raw.dropna(subset=ter_cols).copy()
    data["BreakRisk4Q"] = (data[ter_cols].sum(axis=1) > 0).astype(int)
    data["year"], data["quarter"] = stata_yq_to_year_quarter(data["yq"])
    data["year_c"] = data["year"] - data["year"].min()
    data["quarter_sin"] = np.sin(2 * np.pi * data["quarter"] / 4)
    data["quarter_cos"] = np.cos(2 * np.pi * data["quarter"] / 4)
    data["Affected_x_S_FC"] = data[interaction]
    data = data.dropna(subset=FEATURES + ["BreakRisk4Q"]).sort_values(
        ["year", "quarter", "C_ID"]
    )
    data.attrs["source_file"] = str(path)
    data.attrs["ter_cols"] = ter_cols
    data.attrs["interaction_col"] = interaction
    data.attrs["variable_labels"] = dict(zip(meta.column_names, meta.column_labels))
    return data

def make_mlp(seed=42):
    return Pipeline(
        [
            ("scaler", StandardScaler()),
            (
                "model",
                MLPClassifier(
                    hidden_layer_sizes=(12, 6),
                    activation="relu",
                    alpha=0.01,
                    learning_rate_init=0.003,
                    max_iter=1500,
                    early_stopping=True,
                    validation_fraction=0.15,
                    n_iter_no_change=30,
                    random_state=seed,
                ),
            ),
        ]
    )

def make_logit():
    return Pipeline(
        [
            ("scaler", StandardScaler()),
            ("model", LogisticRegression(max_iter=1000, solver="lbfgs")),
        ]
    )

def model_holdout(cutoff_year=2021, estimator="mlp", seed=42):
    data = load_reorganization_data()
    train = data[data["year"] <= cutoff_year]
    test = data[data["year"] > cutoff_year]
    model = make_mlp(seed) if estimator == "mlp" else make_logit()
    model.fit(train[FEATURES], train["BreakRisk4Q"])
    pred = model.predict_proba(test[FEATURES])[:, 1]
    metrics = {
        "estimator": estimator,
        "cutoff_year": cutoff_year,
        "n_train": len(train),
        "n_test": len(test),
        "auc": roc_auc_score(test["BreakRisk4Q"], pred),
        "average_precision": average_precision_score(test["BreakRisk4Q"], pred),
        "brier": brier_score_loss(test["BreakRisk4Q"], pred),
        "observed_rate": test["BreakRisk4Q"].mean(),
        "mean_predicted_rate": pred.mean(),
    }

```

```

    }
    result = test.copy()
    result["pred"] = pred
    result["residual"] = result["BreakRisk4Q"] - result["pred"]
    return model, train, result, metrics

def rolling_oos_predictions(start_cutoff=2018, end_cutoff=2022, seed=42):
    data = load_reorganization_data()
    pieces = []
    metrics = []
    for cutoff in range(start_cutoff, end_cutoff + 1):
        train = data[data["year"] <= cutoff]
        test = data[data["year"] == cutoff + 1]
        if len(test) < 50 or train["BreakRisk4Q"].nunique() < 2:
            continue
        if test["BreakRisk4Q"].nunique() < 2:
            continue
        model = make_mlp(seed)
        model.fit(train[FEATURES], train["BreakRisk4Q"])
        pred = model.predict_proba(test[FEATURES])[:, 1]
        part = test.copy()
        part["pred"] = pred
        part["residual"] = part["BreakRisk4Q"] - part["pred"]
        part["train_cutoff"] = cutoff
        pieces.append(part)
        metrics.append(
            {
                "train_cutoff": cutoff,
                "eval_year": cutoff + 1,
                "n_train": len(train),
                "n_test": len(test),
                "auc": roc_auc_score(test["BreakRisk4Q"], pred),
                "brier": brier_score_loss(test["BreakRisk4Q"], pred),
                "observed_rate": test["BreakRisk4Q"].mean(),
                "mean_predicted_rate": pred.mean(),
            }
        )
    return pd.concat(pieces, ignore_index=True), pd.DataFrame(metrics)

def parameter_vector(model):
    mlp = model.named_steps["model"]
    arrays = list(mlp.coefs_) + list(mlp.intercepts_)
    return np.concatenate([array.ravel() for array in arrays])

def normalized_distance(model_a, model_b, reference_model):
    vec_a = parameter_vector(model_a)
    vec_b = parameter_vector(model_b)
    vec_ref = parameter_vector(reference_model)
    return np.linalg.norm(vec_a - vec_b) / max(np.linalg.norm(vec_ref), 1e-12)

def stability_diagnostics(reference_year=2018, last_cutoff=2022, seed=42):
    data = load_reorganization_data()
    reference_train = data[data["year"] <= reference_year]
    reference_model = make_mlp(seed)
    reference_model.fit(reference_train[FEATURES], reference_train["BreakRisk4Q"])

    null_distances = []
    for null_seed in range(seed + 1, seed + 21):
        null_model = make_mlp(null_seed)
        null_model.fit(reference_train[FEATURES], reference_train["BreakRisk4Q"])
        null_distances.append(
            normalized_distance(null_model, reference_model, reference_model)
        )
    threshold = float(np.quantile(null_distances, 0.95))

```

```

rows = []
for cutoff in range(reference_year, last_cutoff + 1):
    train = data[data["year"] <= cutoff]
    test = data[data["year"] == cutoff + 1]
    if len(test) < 50 or test["BreakRisk4Q"].nunique() < 2:
        continue
    model = make_mlp(seed)
    model.fit(train[FEATURES], train["BreakRisk4Q"])
    pred = model.predict_proba(test[FEATURES])[:, 1]
    rows.append(
        {
            "train_cutoff": cutoff,
            "eval_year": cutoff + 1,
            "n_train": len(train),
            "n_test": len(test),
            "distance_to_reference": normalized_distance(
                model, reference_model, reference_model
            ),
            "null_q95": threshold,
            "auc": roc_auc_score(test["BreakRisk4Q"], pred),
            "brier": brier_score_loss(test["BreakRisk4Q"], pred),
        }
    )
null = pd.DataFrame({"distance": null_distances})
diagnostics = pd.DataFrame(rows)
return diagnostics, null

def roc_pr_curves(y_true, pred):
    fpr, tpr, _ = roc_curve(y_true, pred)
    precision, recall, _ = precision_recall_curve(y_true, pred)
    return fpr, tpr, precision, recall

def save_figure(fig, stem):
    pdf_path = FIG_DIR / f"{stem}.pdf"
    png_path = FIG_DIR / f"{stem}.png"
    fig.savefig(pdf_path, bbox_inches="tight")
    fig.savefig(png_path, bbox_inches="tight")
    plt.close(fig)
    return pdf_path, png_path

```